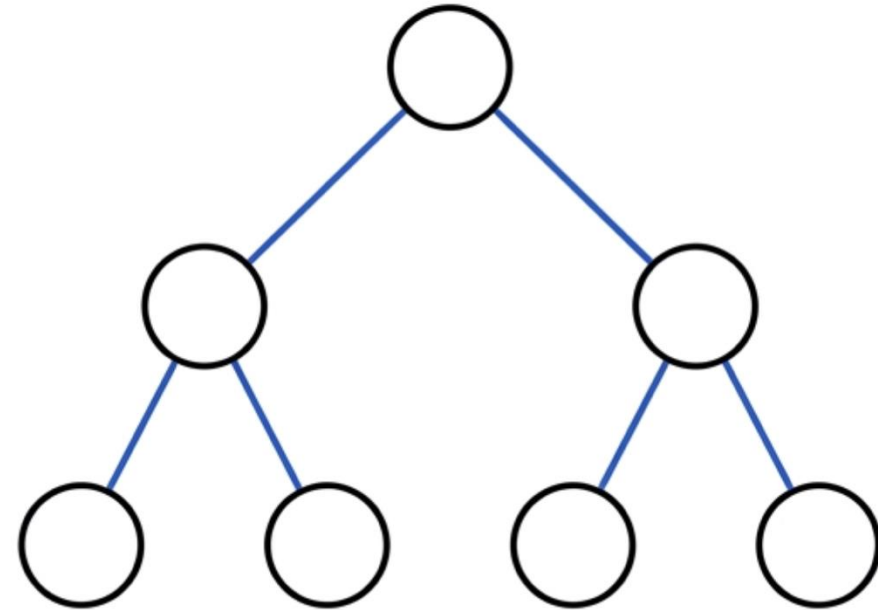


Árboles binarios



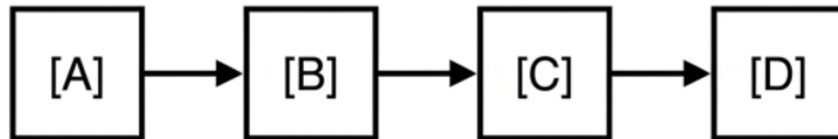
Objetivos

- Comprender la topología no lineal de los árboles binarios.
- Diseñar e instanciar la estructura de un nodo de árbol binario utilizando plantillas (templates) para polimorfismo paramétrico y el paradigma RAII (`std::unique_ptr`) para garantizar una gestión de memoria segura.
- Demostrar que los recorridos en profundidad operan con una complejidad temporal de $\mathcal{O}(n)$

Árboles binarios

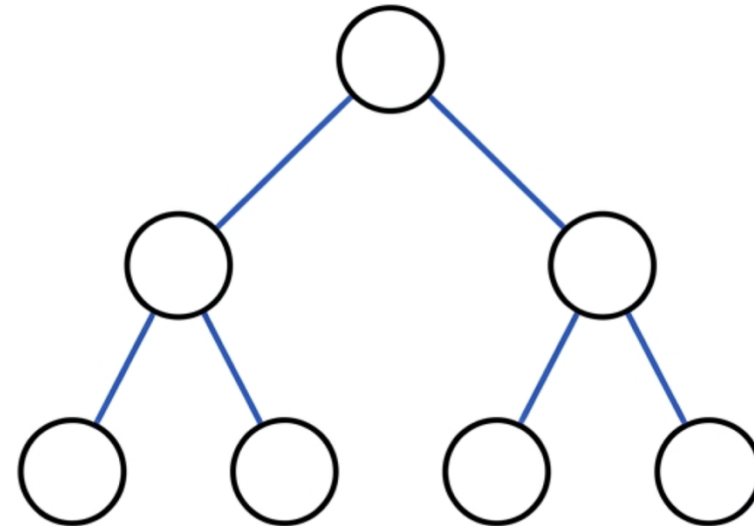


Colección Lineal



Conexión estrictamente secuencial. Implica una búsqueda exhaustiva con complejidad temporal $O(n)$.

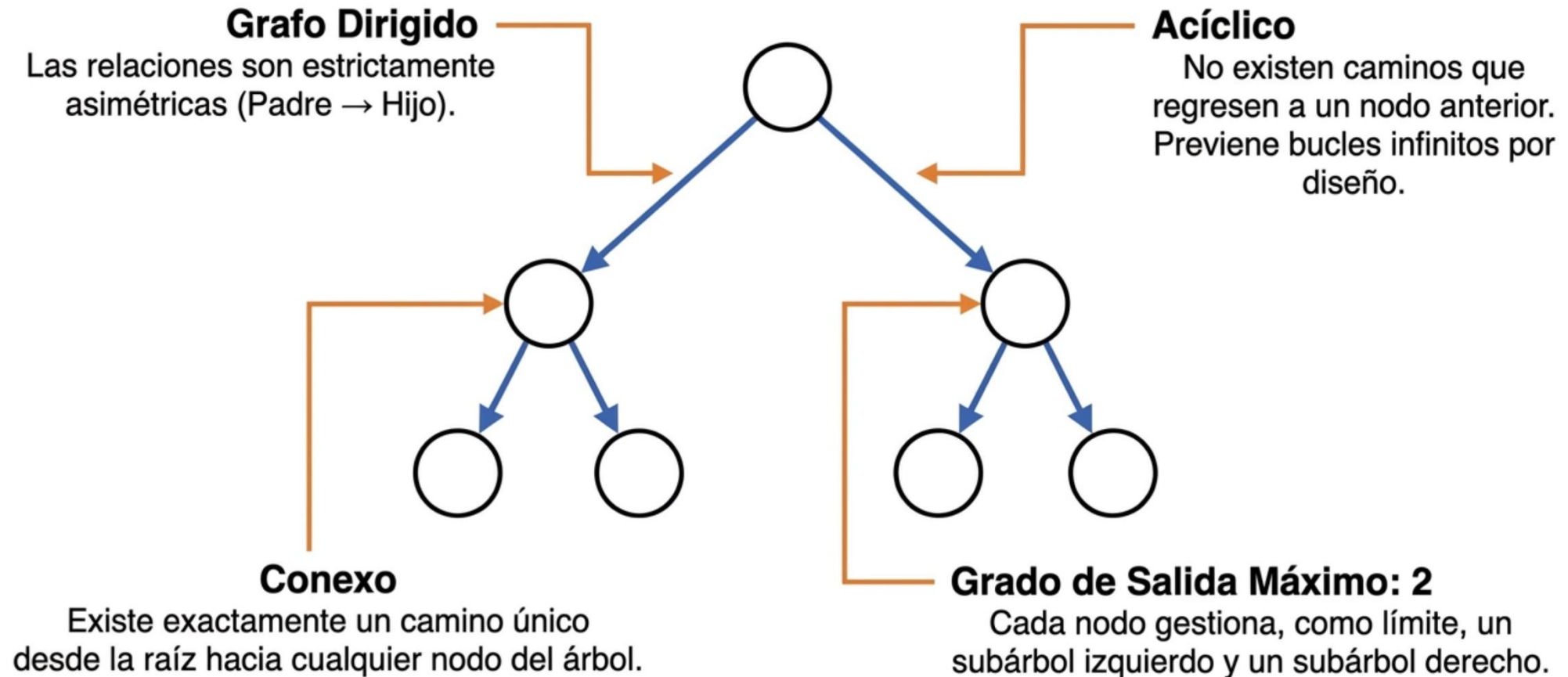
Colección No Lineal



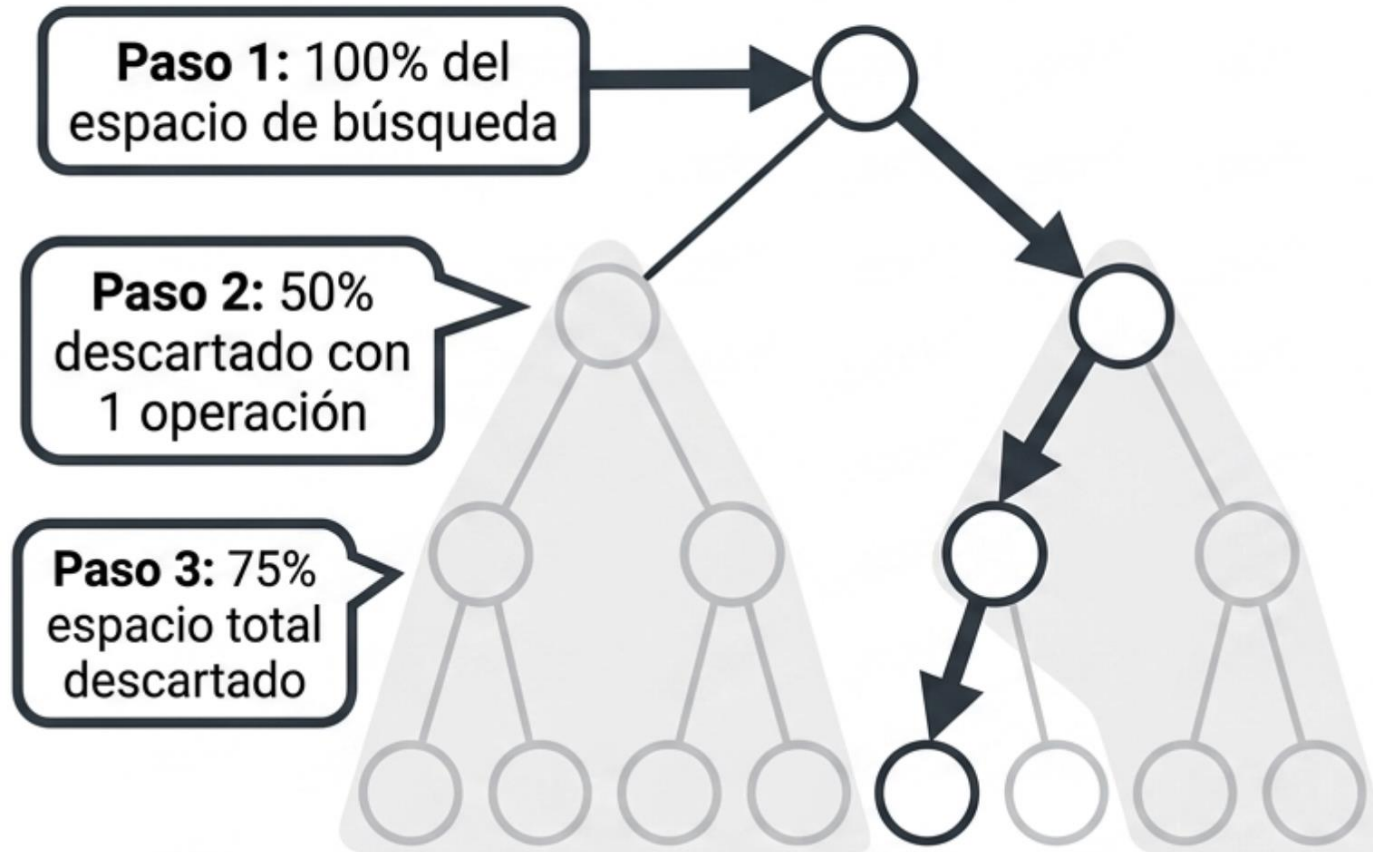
Conexión jerárquica. Ramificación de decisiones que supera la restricción posicional.

En una colección no lineal, superamos la restricción secuencial. Un objeto no solo tiene un 'siguiente', sino que actúa como matriz de múltiples subestructuras.

Árboles binarios



Árboles binarios



$$\text{Altura Mínima} = \lfloor \log_2 (n+1) \rfloor - 1$$

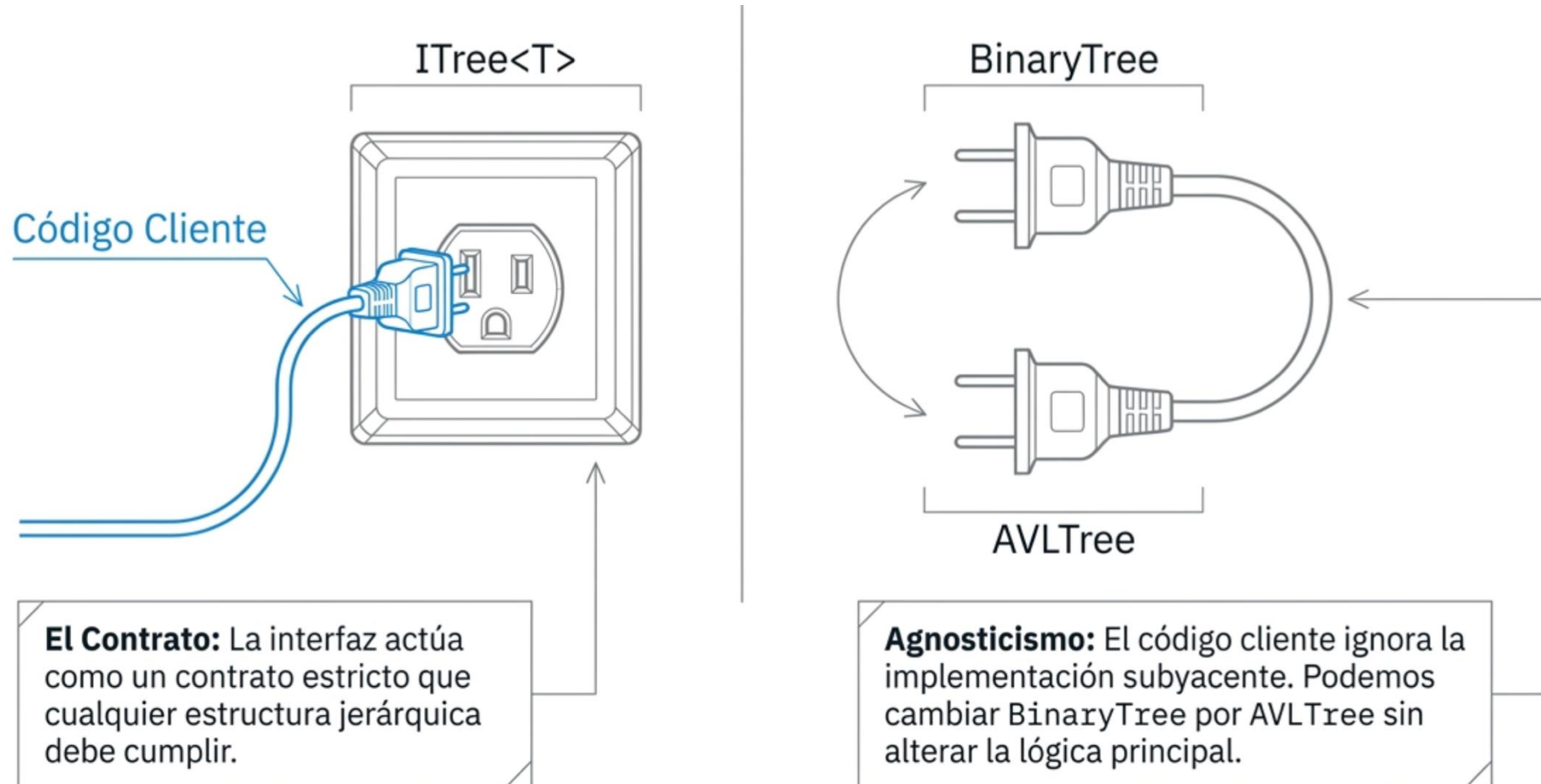
Árboles binarios



Dimensión	C++ Clásico (Legacy)	C++ Moderno (RAII)
Gestión de Memoria	Punteros crudos (Node*) + invocación manual de delete.	Smart Pointers (std::unique_ptr). Propiedad exclusiva de recursos.
Riesgos Operativos	Alta probabilidad de Memory Leaks y Dangling Pointers.	Liberación determinista. Destrucción en cascada garantizada por el compilador.
Sistema de Tipado	Macros dependientes o tipado inseguro (void*).	Templates estandarizados (template <typename T>) para Type-Safety estricto.

Implementar grafos complejos exige seguridad arquitectónica. Abordaremos el Árbol Binario mediante el principio Resource Acquisition Is Initialization (RAII).

Interfaz



Interfaz



```
template <typename T>
class ITree {
public:
    virtual ~ITree() = default;
    virtual void insert(T value) = 0;
    virtual void clear() = 0;
};
```

Abstracción de tipo. Permite estructuras genéricas.

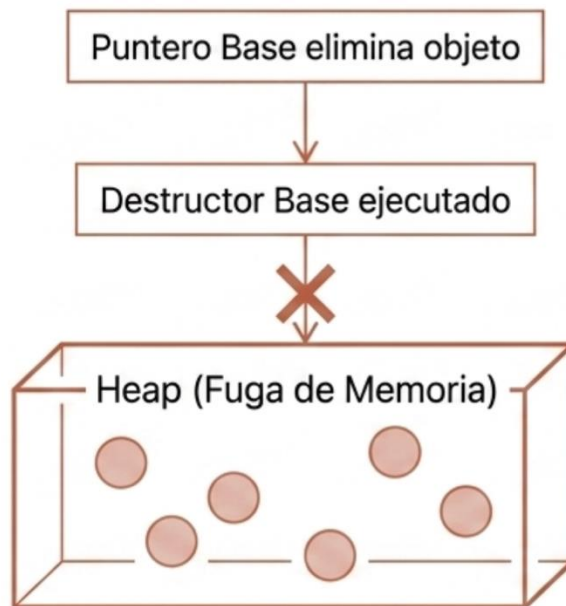
Función virtual pura. Obliga a las clases derivadas a implementar la lógica, convirtiendo a ITree en una clase abstracta.

Garantiza una vía estándar y segura para vaciar la estructura.

Interfaz

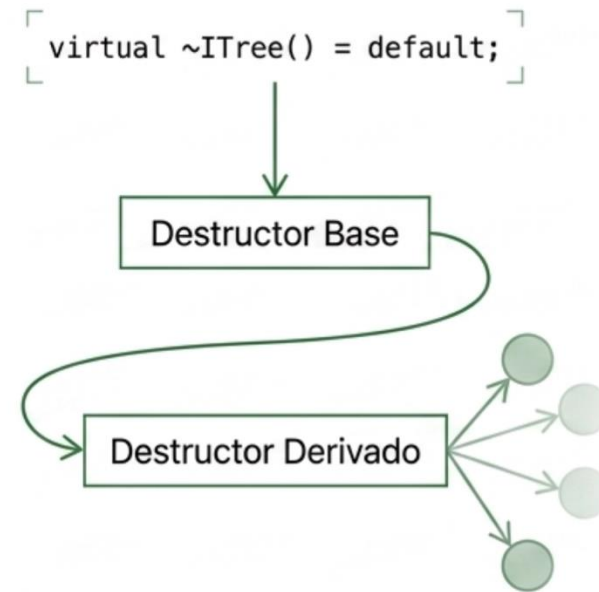
Destruir un objeto polimórfico sin un destructor virtual rompe la cadena de `std::unique_ptr`.

[La Cadena Rota]



El Problema: Si el destructor base no es virtual, el destructor derivado nunca se llama. Los nodos quedan huérfanos.

[Destrucción en Cascada]



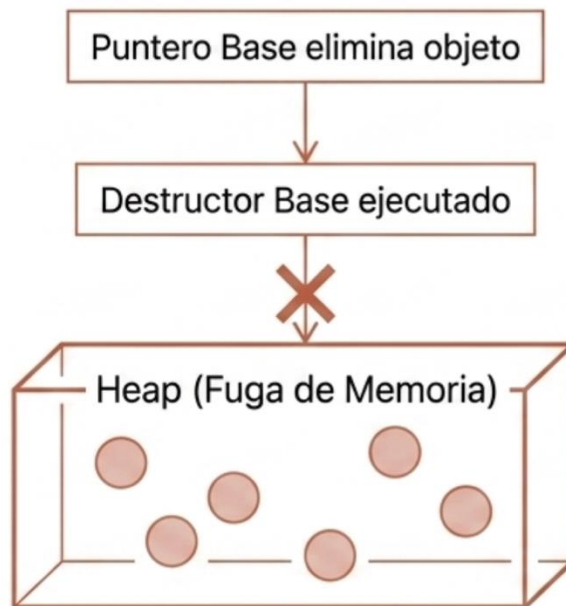
La Solución: Garantizar que la limpieza comience en la clase derivada y suba por la jerarquía.

Interfaz



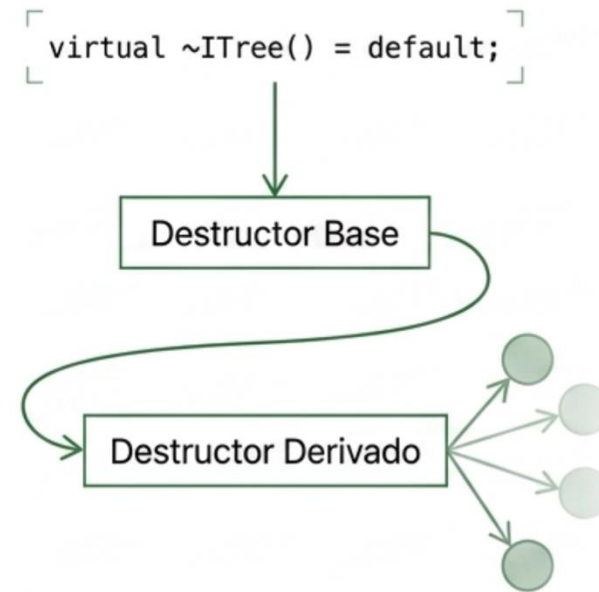
Destruir un objeto polimórfico sin un destructor virtual rompe la cadena de `std::unique_ptr`.

[La Cadena Rota]



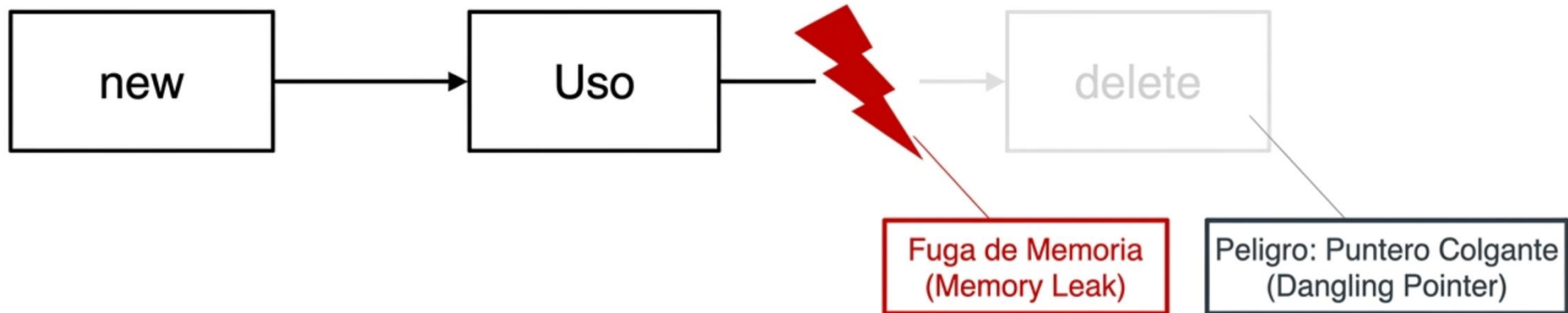
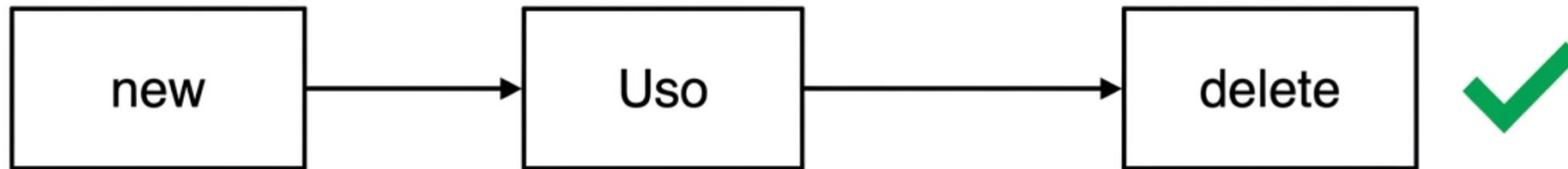
El Problema: Si el destructor base no es virtual, el destructor derivado nunca se llama. Los nodos quedan huérfanos.

[Destrucción en Cascada]



La Solución: Garantizar que la limpieza comience en la clase derivada y suba por la jerarquía.

Manejo de memoria



Manejo de memoria (RAII)



- El ciclo de vida de la memoria dinámica se vincula estrictamente al ciclo de vida de un objeto local.
- Al salir del ámbito local, el destructor se ejecuta de manera determinística.
- Resultado: Liberación automática, cero intervención manual.

Punteros inteligentes



`std::unique_ptr`

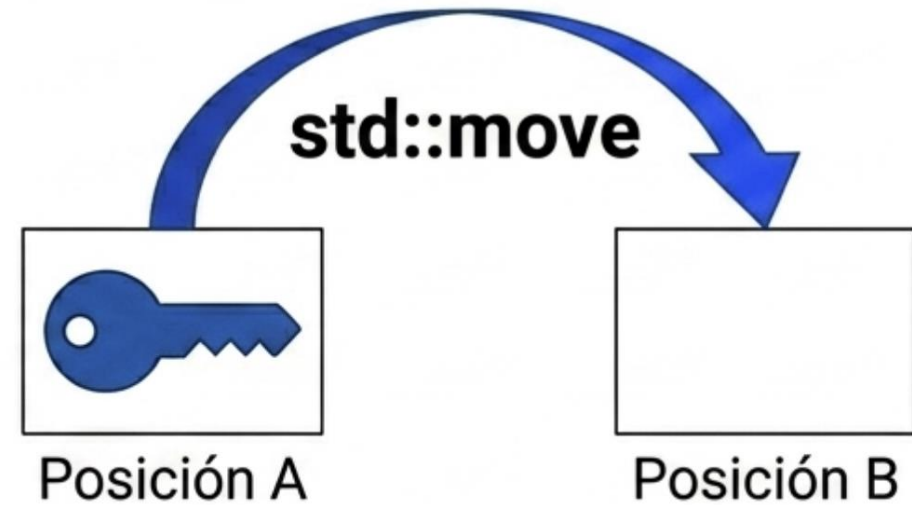
- **Propiedad Exclusiva:** 1 único dueño.
- **Rendimiento:** Cero sobrecarga (zero overhead).
- **Relación Lógica:** Pertenencia estricta.



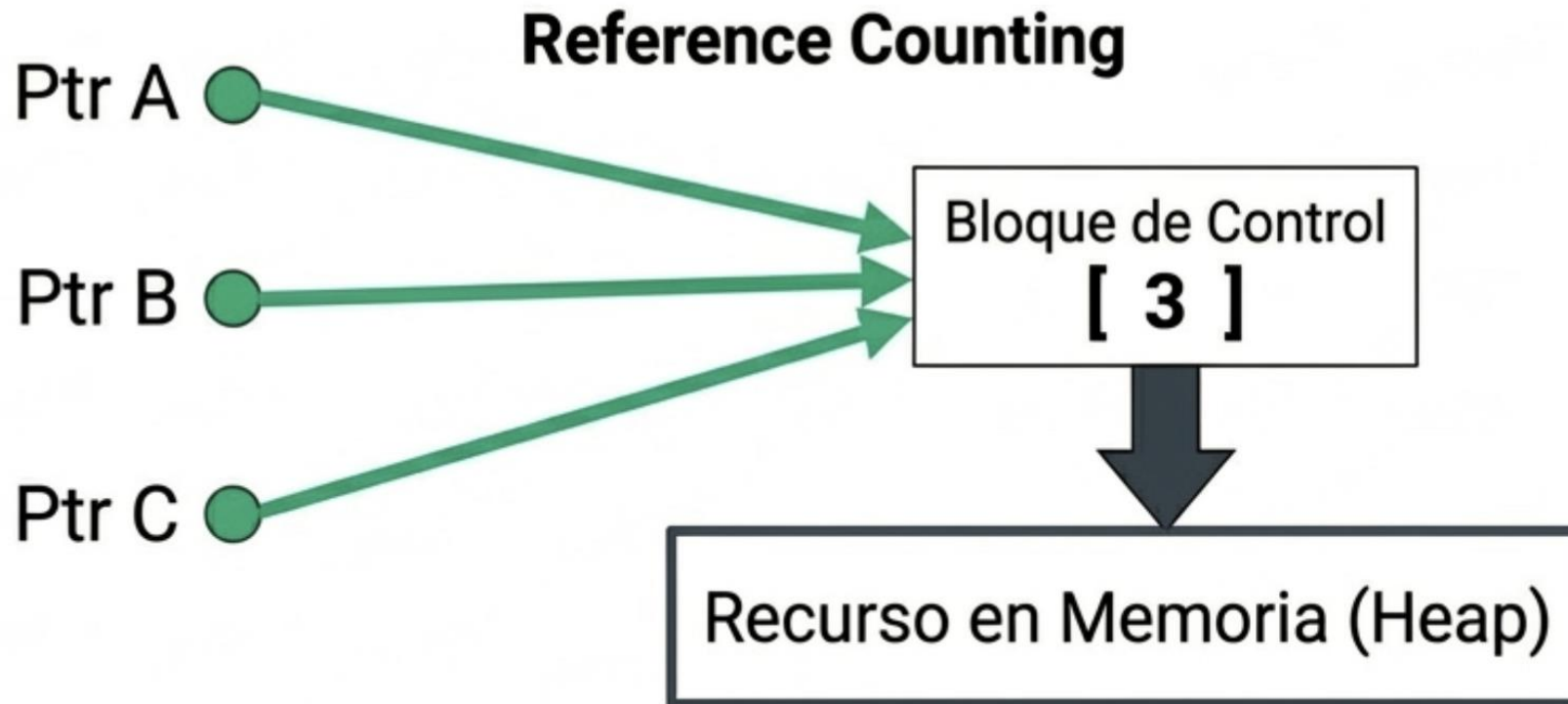
`std::shared_ptr`

- **Propiedad Compartida:** +1 dueños concurrentes.
- **Rendimiento:** Penalización ligera (bloque de control).
- **Relación Lógica:** Grafos cíclicos o sistemas concurrentes.

std::unique_ptr

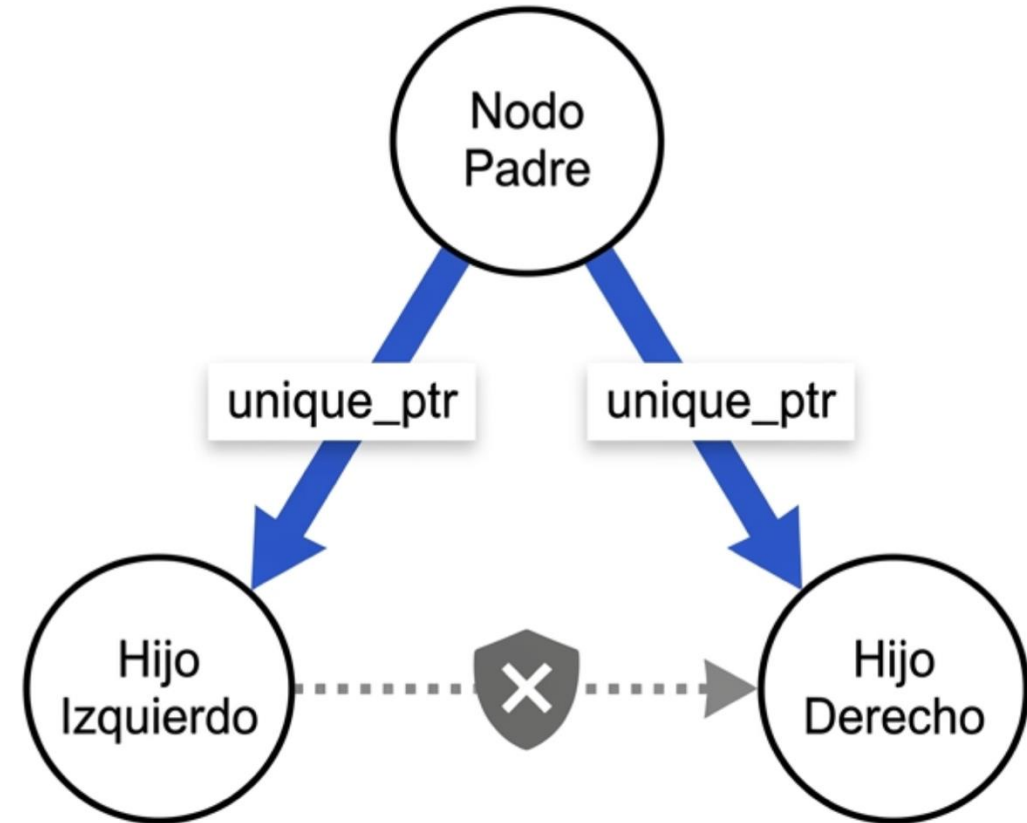


std::shared_ptr



Punteros inteligentes en AB

- La jerarquía de un árbol es matemáticamente estricta: un nodo hijo pertenece a un y solo un nodo padre.
- Al declarar las ramas como `unique_ptr`, expresamos esta pertenencia exclusiva en el sistema de tipos.
- El compilador impide empíricamente compartir nodos de forma accidental, garantizando que el árbol nunca se convierta en un grafo con ciclos.





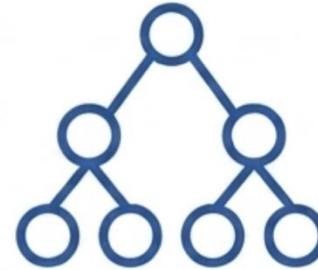
Punteros inteligentes en AB



(RAII)



(unique_ptr)

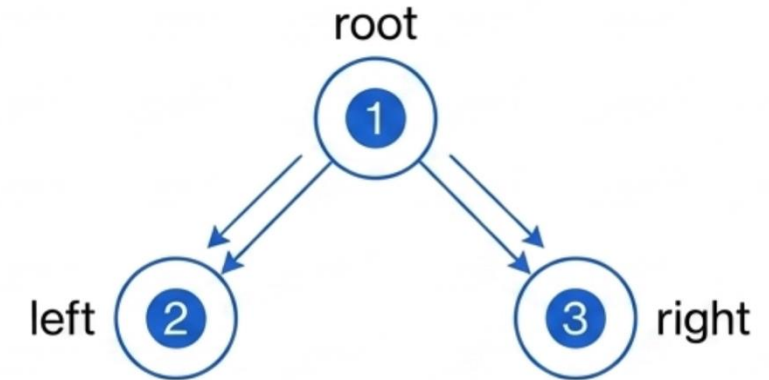


(Topología)



(Arquitectura Segura)

Recorrido pre-order

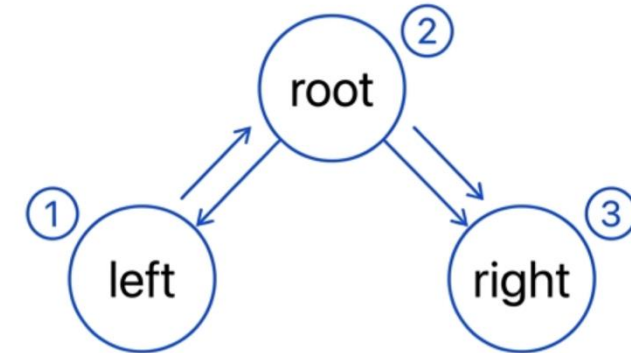


Nodo (N) → Izquierda (L) → Derecha (R)

PREORDER-TREE-WALK(x)

- | | | | |
|---|--|-------|----------------------|
| 1 | if $x \neq \text{NIL}$ | _____ | Condición base |
| 2 | print $x.\text{key}$ | _____ | Procesar Nodo N |
| 3 | PREORDER-TREE-WALK($x.\text{left}$) | _____ | Descenso recursivo L |
| 4 | PREORDER-TREE-WALK($x.\text{right}$) | _____ | Descenso recursivo R |

Recorrido In-order

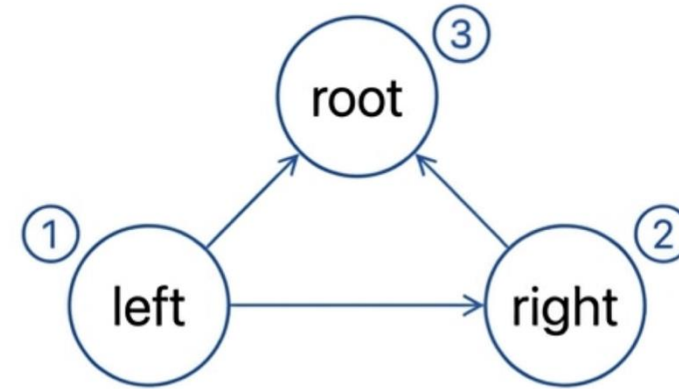


Izquierda (L) → Nodo (N) → Derecha (R)

INORDER-TREE-WALK(x)

- 1 **if** $x \neq \text{NIL}$
- 2 INORDER-TREE-WALK($x.\text{left}$) ————— Agotar profundidad L
- 3 **print** $x.\text{key}$ ————— Procesar Nodo N
- 4 INORDER-TREE-WALK($x.\text{right}$) ————— Descenso recursivo R

Recorrido post-order



Izquierda (L) → Derecha (R) → Nodo (N)

POSTORDER-TREE-WALK(x)

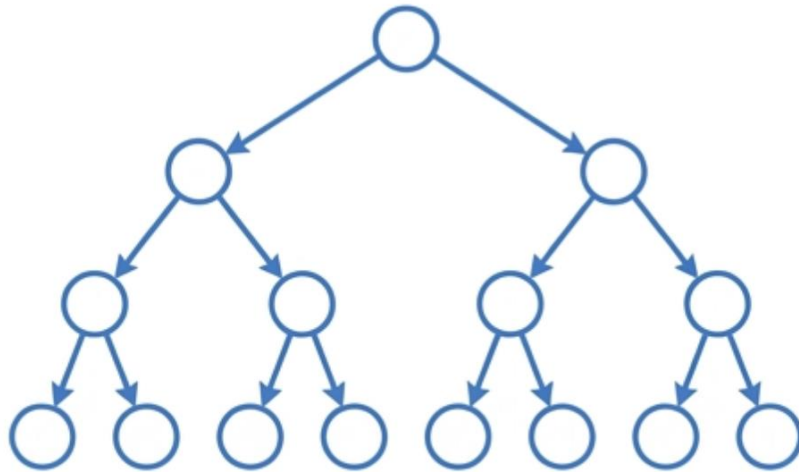
- 1 **if** $x \neq \text{NIL}$
- 2 POSTORDER-TREE-WALK($x.\text{left}$) ————— Agotar profundidad L
- 3 POSTORDER-TREE-WALK($x.\text{right}$) ————— Agotar profundidad R
- 4 **print** $x.\text{key}$ ————— Procesar Nodo N



Recorridos

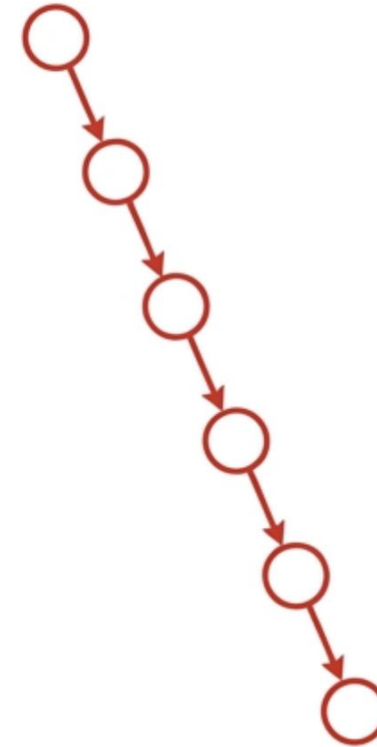
Algoritmo	Topología Exacta	Aplicación Estructural
Pre-Orden	N $\rightarrow$ L $\rightarrow$ R	Serialización y copia de estructura (Top-down).
In-Orden	L $\rightarrow$ N $\rightarrow$ R	Extracción monotónica ascendente (BST).
Post-Orden	L $\rightarrow$ R $\rightarrow$ N	Destrucción de memoria y resolución de dependencias (Bottom-up).

Colapso topológico



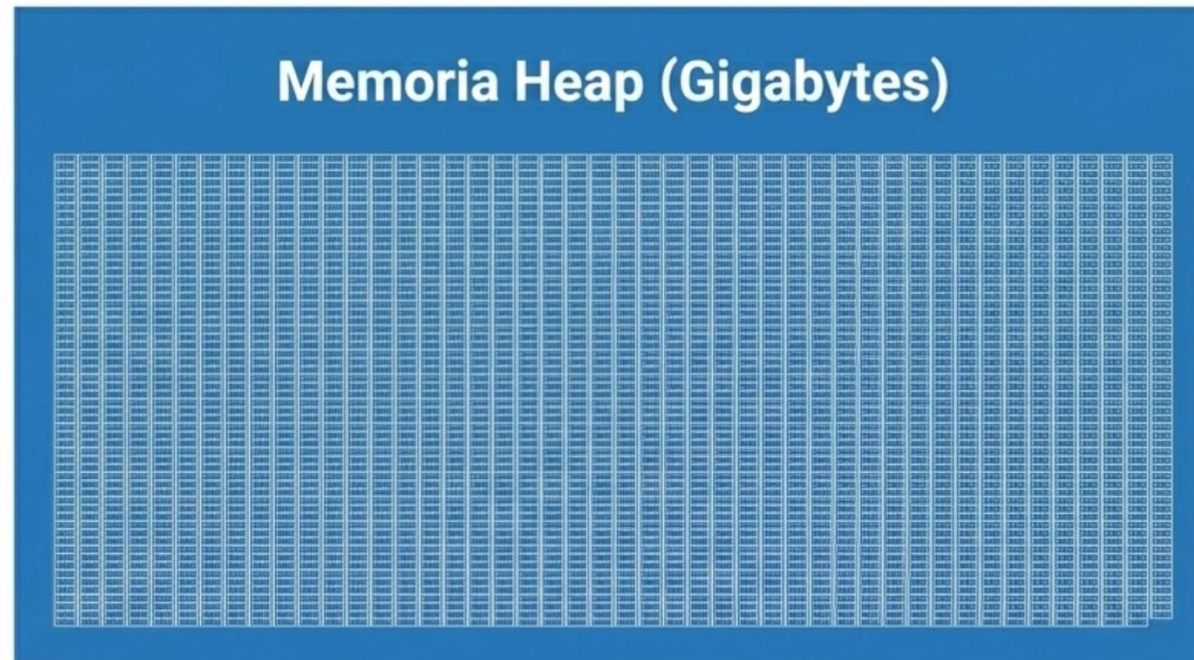
Inserción Aleatoria: $h = O(\log n)$

Insertando datos
pre-ordenados



Degeneración: $h = n$

Colapso topológico



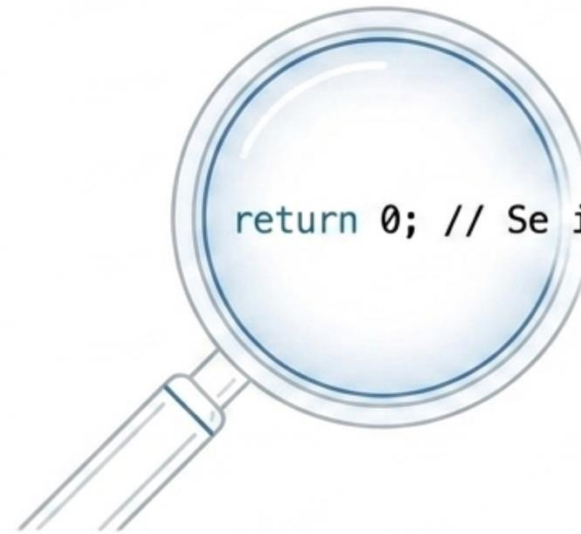
Aloja 100,000 nodos con facilidad
mediante `std::unique_ptr`.



**Call Stack
(~8MB)**

Colapsa (Stack Overflow) al
alcanzar una profundidad
recursiva de 100,000.

Peligro de la gestión RAI



```
return 0; // Se invoca el destructor de root
```

Supongamos que eliminamos la función de recorrido vulnerable. El programa se ejecuta, crea el árbol degenerado iterativamente y llega al final con éxito.

El programa seguirá fallando.

La gestión de recursos RAI limpiará la memoria automáticamente. Pero el destructor predeterminado de **std::unique_ptr** es intrínsecamente recursivo.